GREEN ALIBI — Development Log

This is my working log for GREEN ALIBI. I am testing whether Solar Induced Fluorescence—which is a direct physical proxy for actual photosynthetic activity—can catch crop nd vegetation drought stress earlier than the old vegetation greenness index called NDVI that India's official drought monitoring framework currently relies on for its field data. I kept this log in a plain tracking style, nd it works the exact way I keep every project log in my folder. It is a raw chronological record of my real decisions, dead ends, and code fixes, not some cleaned up summary written after finishing the work. It shows the real messy process.

I organized my txt into separate entries, nd each section covers one specific phase of my desk work: framing the research question, building the SIF nd NDVI extraction pipelines, catching a bad boundary precision problem, bringing rainfall data in for validation, adding a 2nd lag estimation method, running a self review pass, nd expanding my timeline from 3 years to 8. So I did not hide bad files. Every number, correlation math score, nd coding choice below reflects wht I actually found when I ran the scripts, including results tht entirely failed to support my original hypothesis. That's the whole point of keeping it this messy instead of a cleaned up summary.

Index

Entry 1

Am running the GREEN ALIBI project as an independent satellite study. My main goal here is to test if Solar Induced Fluorescence—which works as a direct physical proxy for actual photosynthetic activity—can catch crop nd vegetation drought stress much earlier than the standard vegetation greenness index called NDVI that India's official drought monitoring setup currently uses. I want to see if the government is missing early warnings. See, NDVI is jst a basic reflectance based index. It measures how much visible and near infrared light a leaf's skin reflects back into space, meaning tht reflection only changes once a plant's internal structure has already started rotting or thinning out visibly..... By then, it is already too late. I chose to use SIF because it

tracks a physically earlier, more direct signal inside the leaf. When a plant absorbs sunlight for photosynthesis, a small piece of tht energy is not changed into food but is instead shot back out as fluorescence, a process decided by the quantum yield of photosynthesis. This is the secret. When a plant gets hit by heavy physiological stress, its internal efficiency drops way before its outside greenness changes, nd this drop shows up on my satellite data as a quick change in the fluorescence emission well before NDVI shows even a tiny dip... In short, NDVI shows the plant's fake alibi because it still looks nicely green on top. SIF shows the real physical reality happening inside the cell.

I am looking closely at India's drought declaration rules. The big issue is that government crop insurance cash payouts under the Pradhan Mantri Fasal Bima Yojana scheme depend fully on rainfall deficits and old NDVI based satellite checks. This setup creates a massive bottleneck..... Both tools are jst reflectance or rainfall indicators tht only flash red after the field crops are already dying visibly. Delayed recognition means delayed insurance money, nd farming families suffer heavily because they need this cash fastest right when the heat hits. Am treating this delay as a real, empirical testable question rather than jst assuming it blindly. My main data goal is to check if SIF really catches physiological plant stress way before NDVI, and if tht time lag is big nd consistent enough to matter in real life. If my code proves this, it makes a solid, physics based case to update the warning tools currently used in national policy. Want to show the proof.

So I set a clear goal for my study. Am using satellite derived SIF nd NDVI datasets over drought affected farming districts in India to independently test if SIF shows a measurably earlier drop than NDVI during real drought episodes. I want to calculate the exact timing difference nd check if the resulting time lag is actually large enough n consistent enough to help people with early warning value for drought declaration nd crop insurance timelines. This broke down into 4 clear questions.

1st, I want to find out if SIF drops measurably earlier than NDVI during documented drought onset periods in my study region (RQ1). I need to prove this. Then, I look at the nxt part. So I want to see if this SIF–NDVI lag is identical across different districts nd crop types, or if it varies heavily based on geography n crop physiology (RQ2). It could be random. After tht, I run a government comparison. Am going to check how the timing of SIF based plant stress onset compares against official state drought declaration dates for the same districts nd years (RQ3). Finally, I look at the insurance angle, nd I will calculate if the observed SIF–NDVI time lag gives a practically meaningful early warning window—measured in raw weeks—that is relevant to real crop insurance n drought relief timelines (RQ4). This will complete my tracking loop.

Had 3 main guesses before starting. 1st, I expected tht SIF would show a statistically significant drop well before NDVI during the same dry spell, matching my theory that fluorescence catches a drop in internal photosynthetic efficiency long before you see any outward, reflectance based leaf color loss (H1)... Need to test this pattern across the board. 2nd, I look at crop differences. I guessed tht the total size n exact timing of this SIF–NDVI divergence would not look uniform across crop types because fluorescence behavior depends heavily on unique canopy shapes n species specific photosynthetic pathways (H2). It could vary a lot. Finally, I hve a government timeline guess. I expect that official state drought declarations will lag far behind my satellite SIF stress signal because the official bureau framework relies blindly on slower responding rainfall gauges n greenness indexes instead of checking direct biological plant health metrics (H3). Will check all 3 assumptions against my script outputs when the numbers come in.

So I am starting my earliest planning stage now. My 1st big job is to choose a proper study area with a very clear, well documented history of recent dry spells n reliable government files, so I am looking at candidate zones like the Marathwada nd Vidarbha regions in Maharashtra or the Bundelkhand area tht spreads across Uttar Pradesh and Madhya Pradesh because they all suffer from regular, heavily reported droughts. This will

give me a good baseline. Next, I need to hunt for a usable SIF dataset... I plan to source either the gridded GOSIF product or a proxy layer derived from TROPOMI because the native fine resolution SIF space instruments are simply not built for tracking tiny, individual agricultural fields anyway. This is a tricky data task.

Then I hve to solve the greenness data track. Must find matching NDVI data from MODIS or Sentinel-2 satellites for identical district boxes and time windows. Finally, I need to track down accessible official drought declaration paperwork n any PMFBY insurance payout timing records from local bureaus to use as my real world policy benchmark for my final code comparison. Hve zero files in my project folder right now, nd this log entry simply marks the absolute beginning of my data collection process n I hve not downloaded or achieved any final outcomes yet. I need to start writing my download scripts tomorrow.

Entry 2

Am moving to the next task now. Following my project framing back in Entry 1, my nxt job was to build the actual working satellite pipeline by finding SIF data, sourcing a matching NDVI dataset, and finally producing my 1st real data comparison for Marathwada. This was a heavy coding phase. I finalized my study area nd years. So I chose Marathwada as my primary study region — this was a deep personal choice as much as a methodological one because it is my own home region and everyone knows its well documented recent drought history... I feel connected to this data.

Selected 3 distinct years for my timeline. I chose the season from June 1 to November 1, which means day of year 153 to 305 covering the main Kharif growing season. I picked 2015 nd 2018 because both are heavily documented Marathwada drought years, with 2015 in particular being the bad year of the severe Latur water crisis when trains had to carry water. It was a terrible time. Then I chose 2020 as my last tracking year. This was a comparatively normal monsoon year tht I will use as a clean baseline for my final script comparisons. Timeline's locked in, moving on.

I found my SIF data source. Picked out the GOSIF v2 dataset, which is a global 0.05-degree 8-day solar induced fluorescence product made from OCO-2, MODIS, nd weather reanalysis models by Li n Xiao at the University of New Hampshire. It is the most practical choice. Also I chose this because native space instruments are not built for tiny farm monitoring anyway. Downloaded 60 8-day GeoTIFF files in total — grabbed exactly 20 files each for my 3 target years 2015, 2018, n 2020 to cover my main growing window timeline. The download took some time.

Clipping nd scaling.

So I wrote a python script to crop each global file. I set the script to cut the maps exactly to a Marathwada bounding box between 17.5°N to 20.5°N nd 75.0°E to 78.5°E so it covers all 8 of my home districts perfectly. This step was highly critical... I had to multiply the raw digital values by a scale factor of 0.0001 to convert them into real physical SIF units. Then I had to explicitly mask out 2 specific fill value codes where 32766 means water bodies and 32767 means non vegetated or missing data blocks before running any calculations. Skipping this step would have ruined everything..... These dummy background numbers are numerically huge n sitting far outside the real SIF range, so a naive average would get really corrupted by them... Coded a 2nd script after that. Forced it to aggregate each clipped image into a single district wide mean SIF number per date, nd I also recorded the fraction of valid non masked pixels for every single time step as a data quality sanity check. Data looks clean on my screen now.

Hit a serious cloud masking bug in my NDVI code. I sourced my NDVI data separately using Google Earth Engine from the MOD09Q1 MODIS 8-day surface reflectance product because I wanted its temporal

resolution to match my GOSIF 8-day timeline cadence exactly. My 1st script version was pure garbage. I calculated NDVI directly from raw reflectance bands without applying any cloud quality bitmask filtering lines, nd the resulting NDVI time series came out implausibly noisy on my screen. During the 2020 season, my numbers swung crazy from roughly 0.55 down to 0.14 nd back up to 0.60 within consecutive 8-day windows. This rate of change is physically impossible for agricultural cropland crops. I compared this jump against my SIF data series. My SIF line was smooth all the way through and made complete phenological sense, which proved to me tht my NDVI script was the real problem, not the actual plant health signal. So I found the exact root cause quickly. The error came from heavy cloud and cloud shadow contamination because Marathwada's growing season hits right during the monsoon, n the raw MOD09Q1 product does not reliably exclude bad cloud pixels from its best observation window without explicit quality band filtering code. This messed up my baseline.

So I fixed my code. I rebuilt my entire NDVI extraction script around the MOD13Q1 dataset, which is NASA's own 16-day Vegetation Index product tht runs a Maximum Value Composite algorithm specifically to kill cloud noise. It runs much cleaner now. This file comes with a SummaryQA quality band where 0 means good data, 1 is marginal, 2 means snow or ice, nd 3 is cloudy, so I used this band to explicitly mask out everything except the good to marginal pixels. Then I added 1 more block..... I chose to use a cropland mask using the MCD12Q1 land cover product with IGBP classes 12 n 14. I did this step so that random forest, urban buildings, nd barren rock pixels inside my district boxes would not dilute my crop stress data signal. This step cleaned up my averages nicely. And I chose to trade my old 8-day time resolution for a slower 16-day one — this trade produced a visibly smooth, physically sensible NDVI curve across the chart, which was worth it because a clean 16-day trend line is 1000 times more trustworthy than a noisy, buggy 8-day one. Pipeline's running smooth.

Hit a 2nd, smaller bug. My updated script suddenly crashed inside Google Earth Engine with an Image.clip: Can't transform (0.0,0.0) error string because I was trying to run a .clip() command on a combined image made of 2 really different native resolution layers. One was 250m NDVI nd the other was a 500m derived cropland mask grid. It was an annoying error. So I fixed it easily by removing tht redundant .clip() line from my code entirely, since the reduceRegion() function already restricts its calculation to the supplied geometry boundaries anyway. The clip step was useless nd was the exact thing throwing the error. This deleted the bug.

Merging 2 time series at different temporal resolutions.

Then I had to merge my timelines. Because SIF has an 8-day rhythm nd my new cleaned NDVI uses a 16-day cadence, the 2 tracks did not share an identical date grid on my spreadsheet. Had to change my join logic. Chose to link the 2 series using a nearest date matching loop per year instead of forcing an exact date join, so each 8-day SIF value gets paired up nicely with its closest available 16-day NDVI number on the timeline. The combined data frame finally looks right.

I caught a bad labeling bug. An early version of my comparison plot accidentally labeled 2018 as a normal monsoon year on the chart, when it is actually the 2nd of the 2 documented drought years in my project folder. It was an embarrassing mistake. This was jst a simple conditional logic oversight in my code because only 2015 had been explicitly flagged as a drought year inside the plotting code block. Next I noticed the txt error n corrected it before I drew any interpretations or conclusions from the mislabeled chart. This kept my figures accurate.

Ran my 1st seasonal comparison. With my corrected coding pipeline ready, I plotted the SIF n NDVI trajectories across all 3 target years to check the trends... A visual pattern holds everywhere. SIF always begins dropping from its maximum seasonal peak well before NDVI does anything. My data shows tht NDVI holds near its top peak values for a noticeably longer window of time after my SIF line has already started crashing

down toward zero. This is a real, replicated pattern, and it directly supports my core premise that SIF responds faster to post peak biological changes than old NDVI does. But I must fix my own overclaim here.

Initially made a lazy mistake. When I just eyeballed the 3 charts carelessly at 1st, I claimed tht this time lag was distinctly larger during the 2 bad drought years 2015 and 2018 than in the normal 2020 monsoon year. So I chose to check this claim properly. I calculated each year's drop as a flat percentage of its own peak value at every single subsequent date on the calendar. My strict math did not support my initial guess. The real time lag stayed at a broadly similar size of roughly 15 to 25 days across all 3 years alike. Drought did not amplify the lag gap. I am retracting tht bad overclaim right here in my journal rather than leaving a fake story standing in my log. My correct, defensible finding is that SIF leads NDVI's decline in general. If I want to prove whether drought specifically changes this time gap, I will need to calculate a proper quantitative lag metric like a day of year threshold math test or a formal cross correlation script instead of jst looking at curves. That is my next coding task.

Status after this entry.

So I finished building both the SIF nd NDVI extraction pipelines. Successfully cloud/quality screened nd cropland masked all my raster grids, covering 3 full years of data from 2015, 2018, n 2020 over the Marathwada region. The files are fully clean now [INDEX]. I observed a real, replicated SIF leads NDVI pattern across my timeline charts. But I cannot report this as a final finding yet. I absolutely need a rigorous, numeric lag calculation script to take the place of the lazy visual comparison I used so far — that is my main priority. I also ideally want to download at least 1 additional comparison year to strengthen the small sample size my analysis currently rests on. This will protect my project from easy reviewer questions.

Entry 3

My 2nd log entry ended with an unresolved problem. Ran my very 1st quantitative lag calculation script—comparing how many calendar days after its own maximum peak SIF n NDVI each crossed a series of decline thresholds—but my final results were entirely untrustworthy. The core issue was my tracking window. My chosen observation window from June 1 to November 1 ended way too early before NDVI had even finished declining in 2 of my 3 working dataset years. It was a bad code boundary mistake. Several threshold markers simply never resolved at all in the console because they returned zero crossing dates. The few thresholds that did resolve properly were being averaged very unevenly across different timeline years, which made any comparison between my drought and normal years unreliable rather than just being inconclusive. So I had to scrap that 1st math check entirely.

Chose to widen my timeline window. My fix was to extend the data collection much further into the year by adding 7 extra 8-day GOSIF periods covering day of year 313, 321, 329, 337, 345, 353, and 361 for all 3 years. This pushed my observation cutoff from November 1 out to late December. My old scripts handled this fine. The same clipping and aggregation scripts I built back in Entry 2 picked up these additional 21 files without requiring a single line of code change. Also I got lucky here. My logic was written to scan for whatever files exist inside the raw data folder rather than assuming a fixed date limit. Then I also extended my NDVI extraction script's end date from November 1 straight out to December 31. I re ran the same MOD13Q1 cloud screened, cropland masked pipeline for this longer window. Timeline problem's sorted.

My timeline extension worked perfectly. With this longer window, both my SIF n NDVI seasonal curves now taper down smoothly through day 350 to 360 in all 3 years on the chart. The ugly noise and sharp data truncation are gone, nd more importantly, every single threshold from 90%, 80%, 70%, 60%, down to 50% of the seasonal maximum peak now resolves to an actual crossing date for all 3 years. This is a big win. My earlier

problem of unresolved, missing thresholds is fixed, so I can now run year to year comparisons on a fair, equal footing. My data matrices match up beautifully now.

My final results shocked me. Once I made the timeline comparison fair across the board, my final data did not support my original guess or hypothesis as I had framed it earlier. My 1st hypothesis failed. I found that the mean lag across all 5 thresholds came out to 21.6 days for 2015 which is a drought year, 8.4 days for 2018 which is also a drought year, nd a big 28.5 days for 2020 which is my normal monsoon comparison year. When I averaged them by group, the 2 bad drought years showed a much smaller mean lag of 15.0 days than my single normal year which sat at 28.5 days. This is the exact opposite of wht my H1 theory predicted. This surprise is not an artifact of my older code truncation problem. With my newly extended data window, 2020's time lag at the 70%, 60%, nd 50% thresholds forms a smooth, consistently large trend curve at 36.9, 38.9, and 39.5 days respectively on the chart. It was not an anomalous spike. A single random spike is exactly what u would see if this were still a bad data window error bug. The result looks real, not a fluke.

I hit an additional complication. My calculations showed tht the 2 bad drought years do not behave alike at all because 2015's mean lag sitting at 21.6 days is roughly 2.5 times bigger than 2018's small 8.4 days. This happens despite both being fully documented Marathwada drought years. This wide gap tells me something very clear... Treating "drought year" as a simple binary yes or no label is way too coarse a category for this deep analysis, and it hides the real physical nuances. The 2 droughts definitely differed in their starting onset timing, total severity, or overall duration in ways tht matter deeply for this specific comparison. My current dataset is just too small. A tiny sample of only 2 drought years is nowhere near enough to characterize that variation, let alone average over it in a meaningful way. So I need a lot more timeline entries to fix this gap.

My core finding still holds up. Across all 3 years—drought nd normal alike—I found that SIF consistently begins its post peak decline before NDVI does anything, with NDVI holding near its peak value for a longer stretch after my SIF line has already started dropping. This specific part of the signal is very robust. Replicated it 3 times over in my scripts. This pattern directly supports my project's main premise because it proves SIF is a more temporally responsive proxy for the starting onset of physiological decline than NDVI is. But my data does not support everything. At least not with this current sample, it fails to support the more specific claim tht drought stress heavily amplifies this time lag relative to a normal crop season. I hve to accept this fact. If anything, the limited evidence available in my folder points the other way, though with only 3 years consisting of 2 droughts nd 1 normal season, this is far too small a sample to treat as a confident finding in either direction. And I am just calling it a non result on the drought amplification question [INDEX], not a confirmed contrary finding.

Status after this entry.

The SIF leads NDVI pattern holds up well at this point. I calculated this across 3 separate replicated observations for Marathwada, stretching right across the full growing season all the way through late December. Tracking sheet's in good shape now. The specific guess tht drought years always show a much larger SIF–NDVI time lag than normal years is unsupported by my data sheets. So I found that the 2 dry years differ from each other by a huge margin... In fact, they differ by more than either differs from my single normal monsoon year, which really complicates any lazy binary drought or non drought framing for this comparison. I am choosing to report this exactly as it stands in my console. I will not adjust or tweak my variables to force fit my original thesis guess. The plain, real world result here is a clear replicated SIF leads NDVI pattern with an inconclusive relationship to actual drought severity at this tiny sample size.

Entry 4

I finished my main timeline calculations. After I had a complete, extended window lag analysis safely behind me running from June to December across 2015, 2018, nd 2020 with all 15 threshold year combinations fully resolved, I moved straight onto the big coding task I had been putting off for days. So I had to look at maps. I chose to actually examine my SIF data spatially instead of only looking at it as a lazy, single whole region average number on my spreadsheet. I built my very 1st mapping script called map_sif_spatial.py to plot the raw clipped SIF rasters for a single day of the year at day of year 273, which is right around the exact point where the seasonal decline is well underway in the fields. This script generated an amazing visual..... I forced the program to display the maps side by side for all 3 tracking years using a strictly shared color scale so my comparison stays fair.

My 1st map really looked good. I saw a visibly concentrated low SIF patch marked in dark red sitting right in the west southwest corner of my study area during the 2015 n 2018 grids, while the 2020 panel looked way greener. It was a very satisfying output. This is roughly where I would expect drought stress to pop up if my core theory carries any spatial coherence and not just a temporal one. I am testing whether SIF flags drought earlier n more precisely than old NDVI, so seeing this clear shape gave me high hopes. But then I stopped dead.

Looking closely at that map made me ask a basic question. Realized I should hve asked this much earlier in my logs: what exactly are my study area boundaries? Had been super lazy here. Up to this point, every single script in my project directory—my GOSIF clipping code, my NDVI extraction functions inside Google Earth Engine, all of it—had been built around 1 basic rectangle. I set the geographic limits to 75.0–78.5°E n 17.5–20.5°N right at the start because it seemed roughly like Marathwada's location on a map. I nvr went back to double check those numbers against any precise official shapes, jst eyeballed a general map early on and blindly trusted it for months. Now I had to fix this loose end.

So I opened the Earth Engine Code Editor. I finally typed in the 2 lines that I had nvr bothered to add before—Map.centerObject() and Map.addLayer()—jst so I could draw tht stupid rectangle on my screen nd check it properly. So I zoomed right in on the map. The mistake was obvious. While my box covered all 8 real Marathwada districts, it also swallowed a massive chunk of Telangana including parts of Nizamabad, Bidar, nd Adilabad, touched Solapur, n clipped a corner of Yavatmal. None of these places belong to Marathwada, nd they also do not share my region's drought history because Solapur n the Telangana districts sit in entirely different rainfall regimes. This was a bad data leak. Every mean SIF n mean NDVI number I calculated up to this point had quietly included a huge slice of somewhere else.

So I chose to fix it immediately. I could not just let this go because the claim that this is a Marathwada specific finding is central to my entire project logic. A rectangle is a bad idea anyway, since Marathwada has a very weird irregular shape on the ground with Nanded pushing out far east nd Osmanabad pushing deep south, nd messing with coordinates would not help. Next I decided the correct fix was to drop using a box nd switch to actual official district lines instead. This was the only right way forward.

So I switched to Earth Engine's FAO GAUL 2015 level-2 administrative boundaries dataset. I filtered it down strictly to Maharashtra state nd picked the 8 district names tht make up Marathwada, then merged them into a single precise polygon to replace my old rectangle everywhere in my NDVI script. It was a necessary cleanup pass.

But I immediately found a bug in my own fix. The very 1st time I ran my updated code, I noticed a huge visible gap right where Beed district should hve been on my screen. The rest of the map filled in fine. So I had filtered using the modern district name 'Beed' in my text string, but the old FAO GAUL 2015 dataset stores this area under its older spelling 'Bid', so my code found zero matches. It was an annoying silent failure. The script ran

with zero errors but quietly dropped the district, giving me 7 shapes instead of 8. I only caught it because I looked at the map with my own eyes rather than trusting the terminal's silence. So I fixed the spelling string immediately, nd also added an explicit count check using the .size() function right after my filter loop to print out the number of matched districts. Now, if any dataset naming quirk acts up again, my script will flash a wrong number right on my screen instead of hiding a bad map shape. This makes my check safe.

Fixed my NDVI boundaries 1st. I matched everything perfectly against an academic reference map of Marathwada's district shapes tht I discovered online, but I still had the heavy SIF side left to repair... My code had a huge systematic design gap. My old GOSIF clipping script was running rasterio's window based cropping layout, which is inherently rectangular. A basic rectangle window has absolutely no way to represent or trace an irregular shape polygon boundary like real state borders. It was a clumsy design setup. Exported the precise Marathwada boundary shape right out of Google Earth Engine as a small GeoJSON file to fix this mess. I brought it straight into my local project directory n fully rewrote clip_gosif.py to use rasterio.mask.mask() against tht polygon boundary and not using a loose box window. This step fixed my averages. Every pixel that sits outside the real Marathwada district outlines is now set to a blank NaN value, which removes all the unwanted data that used to leak into my averages from Telangana, Solapur, nd Yavatmal. Glad my regional averages aren't contaminated anymore.

So I re ran my full pipeline chain, triggering clip_gosif.py, then aggregate_sif.py, then compare_sif_ndvi.py, nd finally lag_analysis.py sequentially on my local terminal machine. I processed all 4 scripts in order using my newly corrected shape files, running this heavy pass for both datasets across my entire folder timeline to wipe out the old coordinate calculations. My outputs match up nicely now.

Was bracing for bad changes. I felt really scared tht my new numbers might shift heavily n flip my final thesis conclusion upside down. But tht is not wht happened at all. My quick sanity check SIF means by year came out to 0.137 for 2015, 0.151 for 2018, nd 0.219 for 2020, nd these fresh values beautifully preserved the identical drought versus normal ordering pattern tht I found before cleaning the folder. Trends check out fine. The seasonal SIF vs NDVI curves still showed SIF's decline consistently preceding NDVI's drop across all 3 target years. Also, my time lag numbers told the same story even though they were not identical to my older contaminated box run. Checked the calculations carefully on my screen. Found the mean lag across thresholds was 24.2 days for 2015, a small 5.1 days for 2018, nd 25.5 days for 2020. This means my drought year average of 14.6 days stays much smaller than my single normal year's 25.5 days gap, so my H1 hypothesis is simply not supported again. If anything, the crazy gap between my 2 separate drought years 2015 and 2018 came out even more pronounced with this cleaner geometry than it had with the messy, leaked rectangle. The cleaned up numbers back this up too.

So I am treating this as a truly useful result. Getting the identical qualitative answer from 2 really different, independently built definitions of my study area—a rough rectangle box and a precise administrative shape boundary—is a solid check on whether my earlier trend finding was a real signal or just a silly artifact of sloppy data clipping. It held up beautifully. This does not make my drought amplification hypothesis correct at all. But it does mean I can now say with much higher confidence tht rejecting my H1 guess is not some stupid coding mistake caused by including the wrong pixel rows in my means. The math behind this checks out.

So I still have some big gaps. I do not have any ground truth confirmation tht the specific low SIF red patch on my spatial map actually lines up with farm croplands rather than some other random land cover class sitting inside those same mixed pixels. Also, my sample size stays tiny — the timeline is still stuck at only 3 years of data, which severely limits how far I can generalize any of these findings to other decades. Both of these gaps stay on the record. I refuse to paper over them or tell a fake story.

Set my immediate nxt step. I need to regenerate my spatial SIF map layout using my newly corrected clipped rasters because my boundary fix scripts already overwritten the underlying files anyway. This should be a quick re run. Jst need to trigger my old script execution line again instead of rewriting any new python code blocks, n then I will move straight onto building the interactive Folium version for the dashboard. Ready to keep going from here.

Entry 5

I finished my boundary correction step. And I re clipped my GOSIF layers with the real official 8 district polygon shape rather than using that lazy rectangle nd re extracted my NDVI data the same way through Earth Engine. Next, I wanted to see the new maps. I chose to regenerate my seasonal spatial comparison map for day of year 273 across 2015, 2018, n 2020 because I fully expected it to show Marathwada's actual irregular outline on my screen instead of a plain boring box. But my code failed.

The new map did not change. When I reran my map_sif_spatial.py script on my terminal, the final output image still looked like a smooth, fully colored giant rectangle with zero differences in shape compared to my older bad run. This outcome was highly worrying. So I realized I was facing 2 very different problems — either my district boundary correction step hadn't actually changed anything inside the raw raster data grids, or the clipping worked perfectly but my plotting script was failing to show the transparent mask edges. So I did not want to guess blindly. So I had to look into the raw arrays nxt.

Then I refused to trust my pictures blindly. Rather than guessing whether a good looking or bad looking image map on my screen was right, I chose to build a small, separate python diagnostic script to answer my questions directly. This was a smart code move. Forced the script to open a single clipped SIF data layer, plot it cleanly on screen, n draw my actual official Marathwada boundary polygon shape as a bright blue outline right on top of the matching coordinate workspace axes. It gave me an easy visual test. If the colored raster cells spilled out past that bright blue border line anywhere on the grid, my clipping code had obviously failed. But if the pixels cut off exactly at the blue line edge, it proved my masking logic was 100% correct n my old rectangle layout impression was coming from a different bug. I triggered the script immediately.

The overlay map came back perfectly clean. My SIF data pixels cut off exactly at the boundary line everywhere on the grid, matching all the tight smaller notches n even the complex eastward hook shape sitting right near Nanded. This was a huge relief. Phew. It proved tht my heavy shapefile clipping fix from my last journal entry was really working on the raw grid arrays. The underlying data files inside my folder were correct, n my actual pipeline bug was hiding inside the map plotting script itself instead of the raster layers. Knew where to hunt nxt.

I looked at my script files again. I opened map_sif_spatial.py nd found that while I had fixed clip_gosif.py to mask my raster data nicely against the real district shape file, I forgot to go back n update my main multi panel plotting script to match. This oversight made the screen display look fake. The drawing script was still blindly loading each raster grid using a static, fixed rectangular scale limit left over from my older box based days, n it never actually plotted the district boundary outline lines anywhere on the panels. It was a silly logic lapse. Even though my underlying data array held the correct NaN pixel mask patterns outside the real district shapes, nothing in my plotting code was actually telling the system to make those transparent areas visible to my eyes across 3 side by side charts. It jst filled up the blanks.

I chose to fix the code layout. I fully rewrote the python script to force the tool to pull each raster image's true geographic boundaries directly from its metadata dictionary rather than using a lazy, hardcoded rectangle box string. This change worked immediately. I also added lines to draw the official boundary polygon outline on

every single chart panel right on top of my shared color scale nd the panel spacing formatting patch from earlier. This gave me an excellent clean view. Also chose to rename my output plot file with a fresh version suffix code string. I did this and not silently overwriting my old image file in the directory, matching the same tracking habit I keep using for all my Google Drive exports. This saves me from massive confusion later abt which picture is my corrected one.

My new regenerated figure finally shows the correct map layout. All 3 chart panels now accurately display Marathwada's real, irregular border outlines with the raster pixel data beautifully confined exactly within the district edges... This looks very neat. The core biological pattern itself held up perfectly under this more careful map rendering. Found tht 2015 nd 2018 both show a very distinct, spatially coherent low SIF drought zone heavily concentrated along the western edge of Aurangabad district nd running down straight through the Latur–Osmanabad farming belt. This makes total sense. On the other hand, my 2020 map panel comes out overwhelmingly high SIF green across almost the entire region [INDEX], though there is only a tiny, minor patch of lower stress values sitting way down in the south of my plot. My comparative results look solid.

Learned a huge process lesson from this headache..... A chart figure looking visually plausible or nice on a screen is absolutely not the same thing as it being fully verified. Have to change my workflow mentality. I easily could hve looked at tht 1st rectangular version, lazily decided it was probably fine since the underlying processing pipeline was fixed, n just moved onto the next step. The old picture wasn't obviously broken or flashing red errors, it jst simply wasn't showing what it should have been showing. It was a hidden trap. Building tht standalone overlay test check instead of blindly trusting my own visual read of the image file is wht actually caught tht the map script itself was the remaining stale piece of junk left in the folder. Will nvr trust a silent script again.

So I still hve some massive outstanding gaps to handle. The main issue is tht this map is just a single day of year snapshot at DOY 273 rather than a full continuous seasonal time lapse animation across the months. It is a limited data frame. More importantly, the low SIF drought zone I am seeing on my screen has still not been cross checked against any actual ground level drought impact reports from local district offices. I know tht the visual shapes match what I would generally expect from known historical Marathwada drought geography patterns, but looking consistent with expectations is absolutely not the same thing as being fully confirmed by official field statistics. Cannot stretch my claims. So both of these missing steps go straight into my diary, not me treating this single map layer as more conclusive than it actually is.

Entry 6

I was jst guessing before. I jst took the common talk tht Marathwada had droughts in 2015 n 2018, nd that 2020 was a normal monsoon time, but I nvr checked the real numbers myself for this project. So, to fix this gap before writing the final report, I opened Earth Engine nd downloaded CHIRPS daily rain data for the identical Marathwada area nd the same June to December months tht I already used for my SIF n NDVI work. Needed a real baseline. So, I also pulled 20 years of rain data from 2001 to 2020 (a long time climatology average) because comparing jst these 3 years with each other makes no sense if I want to see wht a real "normal" year looks like.

The results gave me a very good and real proof. Found out tht the 20-year average rainfall for this season is 826.4 mm (the standard deviation is 158.8 mm). Now look at the actual years. In 2015, the rain was jst 648.9 mm, which means it was 21.5% below the normal average... Then 2018 was also bad with 674.8 mm, so it was 18.3% below normal. But 2020 was different and very wet, coming at 1066.6 mm, which is 29.1% higher than normal! So, the drought n normal tags I used before are not jst wild assumptions anymore because my own

data clearly proves the rain shortage. Both 2015 n 2018 had a massive lack of rain, nd 2020 was jst a proper heavy monsoon year.

Then I saw a weird thing. Put these new rainfall figures right next to my old SIF NDVI lag calculations, nd something quite unexpected came up (nd this part does not fit the simple story I was hoping for, but here it is anyway). See, the rain gap in 2015 was -21.5% and in 2018 it was -18.3%, which means they are almost same same with jst a tiny 3% difference between them. But my data for the average SIF to NDVI lag shows a massive 24.2 days gap for 2015, while 2018 had a tiny lag of only 5.1 days! They are miles apart. If less rain was the main reason for a bigger lag, then these 2 dry years should show a similar lag time, but they do not. So, whatever is making the lag act so strangely between these 2 drought times, I cannot jst blame it on how much total monsoon rain went missing.

Will just say it plainly. So I am treating this as a clear limitation of my work instead of making up a fake story to hide it. Maybe the main reason is about when the rain actually failed (like, missing rain in early June versus late September can change how SIF nd NDVI talk to each other, even if the total seasonal water loss looks identical), or maybe the soil was already too dry from the previous year, or maybe it is some other factor my current dataset misses. I just do not have the right data in this project to test these ideas (n I am definitely not going to throw random guesses just to make my project look perfect n clean).

But this extra rain check helps me. Now, the whole "drought year" setup I am using is not jst some copy pasted hearsay from local news, because I tracked n calculated it myself using the same code setup. Also, my old point that "drought makes the SIF NDVI lag bigger" is wrong still stands on a much stronger ground. Now tht I have the actual CHIRPS rain numbers with me, I know for sure that none of my selected "drought years" were secretly getting good rain behind my back.

Entry 7

So I already verified the district rain numbers. But then I decided I needed a proper map view for this rain data, just like the spatial maps I previously coded for my SIF data. Also I did not want jst a boring table with 1 single number printed for each district (that tells you nothing abt the actual geography). I wanted a proper map so I could directly see nd match the exact spots where the rain failed against the specific areas where the SIF crop stress was popping up.

So I coded this map using 3 separate steps. I wanted it to look exactly like my old SIF maps, so 1st I made an interactive Folium choropleth map. For this, I gave colors to each of the 8 districts based on their rain anomaly percentages, nd I added different toggles for all 3 years so a user can hover or click them on their internet browser. After tht, I used GeoPandas nd matplotlib to make a fixed, static version. This one has 3 maps sitting side by side (one for each year) nd I typed the exact rain anomaly numbers directly inside each district boundary because this format works much better for my final college report.

These 2 maps showed a very clear point. Back in 2015, the west n south districts like Aurangabad, Bid, Jalna, Latur, nd especially Osmanabad had very low rain compared to normal, but Hingoli nd Nanded in the east side were almost okay. Then in 2018, the identical west to east difference happened again (but this time it was even more extreme). Aurangabad n Bid both dropped past a huge 35% rain deficit, but funny enough, Hingoli n Nanded actually got extra rain that same year! This is a super important point to think abt... My old total average number for 2018 said the whole region had an 18% rain shortage, which is technically correct on paper, but it hides the real truth tht the drought was mostly hitting jst the west nd south areas while the east side was just chilling. Calling the whole place a "drought year" is a bad generalization because the actual ground reality was really uneven.

Made the 3rd map. This was my main plan from day one because I wanted a single joint picture where SIF maps sit right on the top row and the rain anomaly maps sit directly underneath them for all 3 years. Doing this side by side lets me look at both patterns together (otherwise my brain was hurting trying to remember 2 different maps at the same time). When I put them side by side, the matching spots became super clear. That deep red color showing low SIF in the southwest part during 2015 nd 2018 is sitting exactly over the deep red rain deficit zones on those same years. Also, the matching places tht turned nice n green because of extra rain in 2020 are the same ones showing very high, green SIF health.

Take Nanded district as an example. Its rain barely dropped during both drought times, nd guess wht? My data shows Nanded stayed way greener in SIF compared to the other struggling districts in both 2015 nd 2018. This gives me a big relief because the science actually makes total sense now. Am looking at 2 entirely separate satellite things (one is a faint plant light signal n the other is a rain estimation product from entirely different space sensors) nd yet they are pointing at the same ground spots for the very same reasons.

Then my code broke a little bit. When I generated tht big combined map, I saw a annoying layout bug where the main heading n the 6 small map titles were overlapping n crashing into each other. I solved this layout mess by shrinking the map sizes a bit to give the top txt some extra breathing space (plus I added manual title padding to every single subplot box) because just shifting the main top title higher was not working properly. They needed a clean gap.

Here's wht this actually does and does not prove..... Just because 2 maps look similar to ur eyes does not mean it is a real math test. So I did not calculate any proper correlation or regression equations between my district rain shortages nd the SIF crop values across these 3 years... Doing that would be very bad science anyway (So I only hve 24 total data pairs from 8 districts over 3 years, and they are not even independent data points because neighbor districts during the same monsoon share identical weather). So, my maps are jst a simple, logic based visual match check, n I will write it down exactly like tht without lying or hyping it up.

Entry 8

My Entry 7 ended with a proper limitation. Had tht nice looking dual map of SIF n rain, but I openly said it was jst a visual match because I had not done any real math test on them. But I did not want to leave this gap empty before finishing the whole project (tht would look lazy). I went back n ran the actual math.

Took the files sitting in my `data/processed/` folder—specifically `sif_by_district.csv` and `rainfall_anomaly_by_district.csv`—n combined them using the year nd district names. This gave me 24 rows of data points (which is jst 8 districts multiplied by 3 study years). Rather than relying on jst 1 simple test, I ran a Pearson correlation to check for a straight linear line, n then I also ran a Spearman rank correlation jst in case the link was non linear.

The math numbers came back way higher than I expected. 😃 My Pearson r value was 0.837 (with $p < 0.001$) n my Spearman rho value was 0.857 (also with $p < 0.001$). Both tests say the same thing, n they say it very loudly. This is not a confusing or borderline result hanging near the passing mark. It is a very clear, high proof positive connection showing tht when rain drops, SIF values also drop across these districts n years.

But I am still not going to hype this up or boast too much. The identical warning about data independence from my Entry 7 applies here (jst tht I can explain it better now). See, 24 data rows looks like a good, big sample on paper, but the 8 districts inside a single year are not independent from each other because they touch geographically and share the same regional monsoon clouds. So, the true number of separate events here is actually closer to 3 (jst 1 per year) rather than 24! I will write down the correlation score exactly how

the code gave it to me, without fixing anything, but I am not going to fake it and say I used 24 separate independent data points when I did not.

So wht changes now? My simple "visual check" idea from Entry 7 now has a real, solid number behind it to back it up. But it changes absolutely nothing abt my core result from Entries 3, 4, nd 6 (which proved tht a worse drought does not simply increase the SIF NDVI lag time). This new correlation is jst showing tht SIF n rain values match nicely across different places on the map, which is a separate topic from the time lag question. I do not want to mix up these 2 different ideas just because both of them happen to prove the main point of my project—tht SIF and rain are tracking real, physical connections on the ground.

Entry 9

I found a big mistake. While reading my own paper's Physical Basis part again, I saw a fact abt GOSIF that I wrote down but nvr actually fully thought through. Had written tht GOSIF is not a direct scan from a single satellite, but instead, it is a mixed up statistical model that takes sparse OCO-2 SIF points n glues them together using continuous MODIS light reflection data nd weather reanalysis models. This is 100% correct because tht is how they build the dataset. But I missed writing down the actual scary meaning of this for my project (n this is a major catch): the identical MODIS reflectance data tht GOSIF uses to fill in the blank days between OCO-2 satellite passes is also the main raw data for the NDVI curve tht I am testing it against!

Basically, this means my SIF n NDVI are not 100% separate measurements in this study. On the exact days nd specific fields where the OCO-2 satellite missed taking a direct photo, the GOSIF estimate is actually guessing values using the same MODIS light pictures tht give us our NDVI numbers. (So the 2 datasets are technically copying from each other's notes). So I am not saying my SIF before NDVI time lag answer is fully wrong, n I am definitely not saying GOSIF is a trash dataset. Everyone uses it because raw OCO-2 data points are jst too few nd far between to cover a whole big region like Marathwada. But I need to say this clearly in my notes: a big chunk of matching data is already stuck inside both variables because of the data product I picked..... Right now, I hve zero time n zero extra files to calculate how much of my lag comes from real plant science versus this messy model mixing.

So I am writing this down right now as a straight limitation.

Entry 10

So I felt quite bad about my old writing. Next I reported the massive difference between the 2015 lag (24.2 days) and the 2018 lag (5.1 days) since Entry 3, but when I reread my draft today, I realized I just dropped the fact nd ran away without explaining it at all. That is jst lazy.

Now, looking at it properly, I know I cannot prove the exact reason with my current files. But I can guess some real logical reasons. Both years had almost identical rain shortage numbers, so if total water loss was the only driver, the lag days should look like twins, but they look like total strangers instead! I can think of 3 possible reasons for this massive gap (even though I hve zero data to test them right now)—maybe the rain failed early in June instead of late September, or maybe the big 16-day gap in NDVI satellite photos created a bad timing calculation error in 1 year, or maybe local farmers simply changed their crop types between these 2 seasons. Do not hve any crop calendars or government sowing data to verify this mess, so I am jst writing these down as 3 open guesses and not picking one blindly n acting smart.

Entry 11

I checked everything again. While going through my raw data files and not jst my final txt paragraphs, I found a huge mistake (nd here is what happened, same as the other code mistakes earlier in this log).

Here is what went wrong. Ever since my Entry 4 work, my `data/processed/clipped/` folder had 2 separate files sitting inside it for every single date. So I had the old rectangular crop file named `GOSIF_<year><doy>.tif` from my old broken script, nd I also had the new, correct polygon mask file named `GOSIF_<year><doy>_clipped.tif` from my fixed script tht cuts data using the actual Marathwada border lines. Keeping both files would be okay, but my aggregation script `aggregate_sif.py` was looking for the pattern `"GOSIF_*.tif"` using a regex filter `r"GOSIF_(\d{4})(\d{3})\.tif"`. See the problem here? This regex expects `.tif` right after the date numbers, so every single correct `_clipped.tif` file failed this rule n got skipped silently as a bad name while the old, leaky rectangle files matched it perfectly fine! So, my code was secretly building `marathwada_sif_timeseries.csv` using the wrong, unclipped data this entire time (even though I foolishly thought my Entry 4 nd 5 fixes had updated everything).

Missed this in Entry 4. Back then, I wrote down yearly SIF averages of 0.137 for 2015, 0.151 for 2018, n 0.219 for 2020 as a quick proof check, thinking my border fix was working perfectly. Those numbers are actually right for the old box shaped files (it is jst tht I had no idea I was reading the wrong folder files). My check was just comparing the old bad data against itself under a different name in my head instead of comparing it to the new polygon cropped outputs because my script never read the new files!

But thankfully my district level math was still safe. The script `zonal_stats_sif.py` (which I used for making my map visuals n calculating the SIF rain correlation scores) was not broken at all because that code directly calls the exact `_clipped.tif` txt strings in its file path rather than using a loose glob search pattern. So, my Pearson score of 0.837 and Spearman score of 0.857, plus all my district maps, were nvr wrong to begin with.

So I fixed the script. And I changed the file matching code n the regex txt string inside `aggregate_sif.py` to target only `GOSIF_*_clipped.tif` nd `r"GOSIF_(\d{4})(\d{3})_clipped\.tif"`. This means my code will now strictly load the real polygon cropped outputs from `clip_gosif.py`, even if some old junk files are still sitting inside tht folder. Then, I re ran the full analysis to regenerate my data sheets like `marathwada_sif_timeseries.csv`, `marathwada_sif_ndvi_merged.csv`, `sif_ndvi_lag_by_threshold.csv`, n `final_summary_rainfall_lag.csv` using the proper data (n I also updated the 2 main charts `sif_vs_ndvi_seasonal_v2.png` and `sif_ndvi_lag_by_threshold.png`).

Did the core results flip? Not really. My new fixed lag calculations are 24.3 days for 2015, 6.9 days for 2018, and 28.7 days for 2020 (the old wrong numbers were 24.2, 5.1, n 25.5). The average lag during drought years changed from 14.6 to 15.6 days, while the normal year lag went up from 25.5 to 28.7 days. SIF still drops down before NDVI across all 15 threshold year combinations. My main point—tht drought years actually have a shorter lag than normal years (rejecting H3)—still stands strong on the correct border data. But I should highlight tht the 2018 lag number jumped the most from 5.1 to 6.9 days (tht is roughly a 35% change!), which is important since it is the smallest nd most striking lag value in my whole project.

Need to clean up. I still have those old, unclipped rectangle files sitting inside my `data/processed/clipped/` folder right nxt to my correct data. Chose to leave them there for now instead of deleting them in a hurry because secretly wiping data during a project review looks very fishy. But I must delete them before finalizing this GitHub repository. (Their presence is the exact reason this annoying bug stayed hidden for so long!). If anyone writes a new script later tht uses a loose `GOSIF_*.tif` search pattern, it will break again the same way until those bad files are gone...

Entry 12

Used only one logic till now. Every single lag value I wrote down in this project, even after my Entry 11 fix, comes from the same threshold crossing method. But this technique has a fixed internal bias because it only tracks when the actual plant decline begins (especially when looking at high numbers like 90% or 80%). It cannot tell me if my main results will stay same if I ask the question in a different way. So, before closing this project for good, I decided I needed a solid 2nd opinion from a fresh approach.

I chose time lagged cross correlation. 1st, I took my SIF nd NDVI numbers, changed their scales to 0-1, and focused only on the downward slope months after the peak growth. Then, I put them on a shared daily grid, subtracted the average values to de mean the lines, n checked which day shift gives the best matching correlation score. This test asks a unique question compared to the old threshold setup (rather than catching when the downfall starts, it finds the best single time shift to match the whole shape of both charts). If these 2 different math styles give the same story, then my project results are truly solid.

I caught a huge mistake right before it ruined my data..... In my 1st test run, I allowed my code to search for time lags all the way up to 60 days, but then I noticed tht 2 of my 3 study years had their peak correlation scores sitting dangerously close to that 60-day limit (specifically, 2020 showed a peak at day 56 out of 60). This is a classic fake result tht happens in cross correlation math (if you test a lag tht is almost as long as your entire data timeline, u hve too few overlapping days left to get a trustworthy score, so the peak u see is just the search window running out of space rather than a real best match). To fix this, I capped the max allowed lag to jst N/4 of my total downward slope period (which is a standard math rule, giving abt 32 days here) n then I re ran the script. Now, the peak lag days for all 3 years sit safely inside the middle of my search range instead of getting stuck on the extreme edge, so this is a real best match spot n not just a code artifact of where I stopped looking.

The new results came out. I found the best cross correlation lag points at 13 days for 2015, 4 days for 2018, nd 7 days for 2020 (all 3 are positive numbers, they show super high correlation values of 0.99, 0.99, and 0.99, and none of them got stuck on my search cutoff limit). Because every single year gives a positive lag day, my new separate method independently checks out nd proves the main point of this whole project: SIF definitely drops down before NDVI across all 3 years. This makes my H1 idea way stronger than before.

But I will not lie or fake the parts tht did not match perfectly. See, my old threshold script ranked the lag sizes from biggest to smallest as 2020 (28.7 days), then 2015 (24.3 days), n lastly 2018 (6.9 days). But this new cross correlation script ranks them as 2015 (13 days), then 2020 (7 days), n lastly 2018 (4 days). (At least both math styles agree 100% tht 2018 has the smallest lag of all, so tht part is doubly proved!). But they fight each other on whether 2015 or 2020 has the absolute biggest lag, which is the exact base comparison for my whole H3 drought amplifies lag debate. The simple version: the basic idea that SIF drops before NDVI is now highly trusted because 2 different math styles show it, but the exact day counts and the ranking of the years are still fuzzy because 2 equally valid methods give different orders. Am putting both statements clearly in my final paper n inside this log.

My original threshold numbers do not change at all. I am not replacing my old calculations at all. This new test is just an extra bonus check (like looking at the same map from an entirely different angle) to give my project more depth.

Entry 13

So I sat down to critically review my whole project folder today. Once my paper drafts, project journal, n dev logs were all in a stable state, I went back through my code work with fresh eyes the exact way I would expect a tough thesis committee to do, and not jst lazily treating "it runs n the numbers look right" as the final finish

line. I wanted to find my own mistakes 1st. Going back through my folders turned up the identical list of gaps every single time. I found that I had zero uncertainty quantification around my lag estimates, my district level correlation's non independence was jst asserted blindly without a real measurement, my Google Earth Engine data acquisition step was still unscripted, n—the one that bothered me the absolute most—my RQ3 question had nvr actually been answered anywhere in the final paper draft. It was an embarrassing list of omissions. This log entry records exactly wht I did to solve each of those holes, nd it also notes down the messy code logic tht I tried nd abandoned along the way before getting things right. Not hiding any of the false starts here.

My 1st bootstrap attempt failed. Initially tried running a moving block bootstrap loop directly on my interpolated daily SIF nd NDVI time series datasets by chopping each year's seasonal decline curve into overlapping 10-day chunks. My plan was simple. I chose to resample those blocks with replacement, recompute the cross correlation lag on the shuffled data frames, nd repeat the whole thing 2,000 times on my machine. Every single confidence interval collapsed down to a flat [0.0, 0.0] marker..... This was not a tight range at all, but jst a broken degenerate artifact in my output terminal... It was an actively misleading result. If I had reported this number blindly as the lag is exactly zero days plus or minus nothing, I would be cheating.

So I looked into why it broke. The real reason is tht these seasonal decline curves are strongly non stationary because they trend monotonically downward across the full monsoon harvest months. Shuffling chunks of a trending line really destroys the pattern itself. My resampled time series stopped looking like a real plant decline curve at all, so my cross correlation script just grabbed whatever junk was left over in the array. The loop was jst picking up random noise centered near zero lag. I realized I was jst measuring wht happens when u erase the crop signal, instead of sampling the real uncertainty inside that signal. Had to change my approach immediately.

So I used case resampling instead. I stopped shuffling random data points on my interpolated daily grid, and chose instead to resample exactly which of my original 17 sparse 8-day satellite observations went into building tht target year's curve with full replacement, then re interpolated n re ran the identical cross correlation script from Entry 12. This step fixed the math nicely. This change preserves the actual shape of the seasonal decline curve in every bootstrap replicate while still capturing actual physical uncertainty, and it mimics real life — specifically, it tracks the uncertainty of not knowing in advance which cloud affected overpass dates u will actually get from the satellite during a messy monsoon season. My new results make perfect sense.

So I checked the new ranges on my screen. Found 2015 gave 13 days with a 95% CI of 5 to 20, 2018 sat at 4 days with a 0 to 9 range, n 2020 hit 7 days with a wide 0 to 32 boundary [INDEX]. The big catch is that every pairwise comparison between years overlaps on the timeline, so my exact ranking discussion from Entry 12 is not something my data can support with super high confidence. But my core direction remains safe. My lag calculations stayed non negative in 100% of my bootstrap replicates for every single year, nd it was strictly positive in 84.5% to 99.9% of them. This is a double sided finding: a real quantified strengthening of my H1 hypothesis, but also a real quantified weakening of my confidence in the specific day numbers. Both facts are true at the same time. I wrote them down exactly like this in my paper draft rather than keeping a single point estimate tht looked way more precise than it actually was on the grid. That's the review pass done.

So I ran a spatial test today. I wanted to quantify how much my district level correlation numbers lack independence by using a formal Moran's I metric script, since my old GA_Research_Paper.md draft always stated tht the 24 district year observations backing my r = 0.837 correlation are not fully independent because the 8 districts sit right nxt to each other on the map. This was a basic blind assertion. I chose to fix this by

building a proper Queen contiguity spatial weights matrix directly from the actual FAO GAUL district polygon boundaries with a calculated mean of 3.25 neighbors per district. Then I executed the main code logic... I ran my Moran's I check using a heavy 9,999-permutation significance test for both my mean SIF data and rainfall anomaly tracks separately for each calendar year. My script outputs gave me a massive reality check.

Every single check came back significant. All 6 year by variable combinations returned a p value below 0.05 with the actual I value ranging cleanly from 0.26 to 0.55 against an expected value of −0.14 under spatial randomness. Numbers hold up fine. This does not change my final correlation coefficient number or invalidate my project data because the underlying spatial pattern is real and cross validated against my SIF charts in Section 4.3. But it changes how I count my rows. My data proofs mean I can now state precisely tht my effective sample size is actually closer to a tiny 3 instead of a big 24, rather than just waving my hands at the issue. This fixes my statistical claim.

So I scripted the Earth Engine data download step today. One caveat abt wht this scripting step actually means: the NDVI, cropland mask, nd CHIRPS rainfall extractions were originally all run by hand inside Earth Engine's browser Code Editor right from the start of this project, nd every single version of my Limitations section has explicitly said so. Wanted to automate this messy workflow. Wrote a fresh python file called src/acquisition/gee_data_acquisition.py using the earthengine api package n geemap module. My new code reconstructs exactly wht Section 3.1 of my paper describes by setting up SummaryQA filtered MOD13Q1 bands, the MCD12Q1 cropland mask with classes 12 n 14, CHIRPS seasonal rainfall totals, nd the 20-year climatology baseline all running against my official FAO GAUL Marathwada district polygon. This is a clean script layout.

But I want to be super precise abt wht this file does nd does not fix. I do not hve an authenticated Google Earth Engine account inside this cloud terminal environment right now, so I hve not executed this script nd cannot confirm with my own eyes that it reproduces my original interactive browser output exactly. It is an unrun backup file. What it actually does is turn a purely prose described, only mentally reproducible manual step into a checked in, version controlled repository artifact tht anyone with real Earth Engine API access can easily run nd inspect. Chose to write this exact distinction explicitly into my paper's Limitations text box rather than letting the mere presence of this python file imply more than it has actually earned. So I will not hide this setup detail. Repo's updated with the new file.

I finally answered RQ3 today. This is the part of my original notes I feel worst abt missing, because RQ3 n the research questions list in GA_Project_Report.md always framed my policy motivation partly using the official drought declaration timeline rather than jst a simple rainfall deficit score. But my section 4 inside GA_Research_Paper.md forgot to run this check. My older section 3.5 only looked at a rainfall deficit comparison which answered a real question but missed the government timeline angle. Had to fix this gaping hole. I ran a web search to find a verifiable official Maharashtra drought declaration date for each study year in my folder. For 2018 this pass was super straightforward n well documented, since the state government officially declared a massive drought in 151 talukas across 26 districts including all 8 of my Marathwada study districts on 31 October 2018, based on matching Zee News n EPW news articles from that week. This gave me an exact match.

But my 2015 data search failed. Did not find a comparably clean single date for 2015 because media coverage of the 2015 Marathwada drought is heavily dominated by the subsequent 2016 Latur water crisis reporting rather than a properly dated 2015 Kharif declaration paper. It was a messy archive hunt. Also I chose to report this timeline comparison for 2018 only and stated this data limitation explicitly in my txt rather than picking

some plausible sounding date tht I couldn't actually verify with a clean link source. I refuse to fake my logs. At least the timeline comparison isn't overstating things anymore.

I looked at the actual 2018 numbers. My data shows SIF crossed its 90%-decline threshold on 8 September 2018, NDVI crossed it on 12 September 2018, nd the official state declaration came out way later on 31 October 2018, which means a massive delay of 53 nd 49 days respectively. Sitting with these numbers changed my mind. Realized tht the SIF vs NDVI time gap which is the whole point of my paper is jst 3 to 4 days on the grid. But the satellite to government gap is roughly 7 full weeks. These 2 scales are really different. This RQ3 comparison needs me to say this clearly in my paper: the bigger finding is that any satellite signal beats the current 7-week plus institutional process by a mountain of time. SIF is jst a neat extra way to optimize that big jump. I would much rather state my policy case at the real size it earned instead of letting my narrower technical metrics imply a fake giant impact to the reader.

My core baseline numbers didn't change. Every single update I threw into this entry is either a precision improvement like bootstrap CI or Moran's I metrics, or a truly new comparison like RQ3 tht I definitely should hve run right from the absolute start of my project. I am jst tuning things up, nd my claims are now written down with much more appropriate n modest confidence levels than my older hyped drafts showed. That rounds off the research setup for now.

Entry 14

My paper's Limitations section looked no different across every single version. I kept repeating tht 3 years consisting of 2 droughts nd 1 normal season was way too small a sample size nd tht my 2 drought years differed substantially from each other. So I got tired of jst talking abt this gap, nd decided to actually fix this issue and not just lazily stating it as a permanent limitation block in my txt. Added 5 more years to my folder. Grabbed data for 2016, 2017, 2019, 2022, n 2023. This big expansion step brings my entire satellite study to 8 years total across my timeline sheet. It makes the data much stronger.

1st step: getting the raw data.

I checked the database availability lines 1st. I found that GOSIF v2's 8-day product covers everything from 2000 to 2024, so all 5 of my new target years exist cleanly in their database. I sat down to write the acquisition files. I wrote a new script called src/acquisition/download_gosif_new_years.py to pull them in using the same DOY-153-to-361 seasonal window I already used for 2015, 2018, nd 2020. So I kept the identical file naming convention... This choice means my old clip_gosif.py script requires zero code edits to pick up n crop the fresh files automatically. Then I updated my Google Earth Engine pipeline. I extended the STUDY_YEARS list array inside src/acquisition/gee_data_acquisition.py to all 8 years. Then I also added 2 brand new CSV export code blocks, choosing to download region mean rainfall and by district rainfall tables directly as proper CSV files because my original script only printed raw rainfall totals straight to the terminal console. That old style was fine for typing 3 numbers by hand but it fails badly when u hve 8 years to track. Download setup is ready to go.

Hit a big surprise while setting this up. Cannot reach the main GOSIF data server directly from this cloud workspace environment because it throws a nasty 403 Forbidden code error on every single request, browser user agent header or not. It is a complete network lockout. This is the same kind of nasty IP range block tht stopped Sentinel Hub when I was working on my DOUBLE_JEOPARDY project folders. So I had to alter my system plan — the script download has to run directly on my own home machine hardware, which is the same way tht other project's satellite data pulls did. It is an irritating extra step.

I want to be upfront abt wht is not yet done:

Neither script has actually been executed yet. My raw GOSIF download needs to happen from a normal home internet router line, n my GEE extraction code requires my authenticated personal Earth Engine account. And I hve not triggered the code loops, since this is the identical login requirement the original unrun acquisition script already carried in its file headers. My file metrics are still old. Every single number inside this project workspace as of this log entry is still the original 3-year result from my 1st pass. This entry jst marks the absolute beginning of my expansion work. The very nxt entry will cover what actually came back once both scripts finish running n I re process the full analysis pipeline against 8 years instead of 3. I need to run my clipping, lag analysis, cross correlation loops, bootstrap confidence intervals, n Moran's I scripts again. This will update every single downstream document in my folder. Picking this back up tomorrow.

Entry 15

I ran download_gosif_new_years.py 1st. All 135 files representing 5 years multiplied by 27 composites came down clean on my very 1st attempt because this ran directly from my own internet connection as expected. I did not face any old GOSIF blocking bugs here. My code script worked perfectly. But gee_data_acquisition.py was a whole different story. Every single one of the nxt 3 problems I hit turned out to be an actual bug inside code blocks tht had nvr actually been executed before this pass. So I was testing fresh code path logic here.

Bug 1 — no registered Cloud project.

So I hit a major initialization block right away. Modern Earth Engine now strictly requires u to type ee.Initialize(project=...) using a registered Google Cloud project ID string inside ur setup. My script only carried a bare ee.Initialize() line nd a commented out ee.Authenticate() command, since both lines were dusty leftovers from old days before Google added this strict cloud project requirement to their API. I had to fix this setup line. I added both parameters properly using my own personal EE project ID string.

Bug 2 — MODIS clip against the wrong projection.

My nxt task failed with a bad error message. My green NDVI extraction crashed with a nasty projection transform error while trying to clip a MOD13Q1 image sitting on its native sinusoidal grid SR ORG:6974 against my Marathwada boundary polygon shape in EPSG:4326. It was a total mess. Some specific district boundary geometries jst do not transform cleanly onto tht curved raster edge line. I looked into the code functions. Found tht the .clip() call was redundant anyway, since the reduceRegion() function call right after it already restricts the mean calculation loop to the same geometry limits. I deleted tht useless clipping line from my script file. Runs clean now.

Hit Bug 3 nxt. I noticed a whole district went silently missing from my spreadsheets — even though the code finished running with zero terminal errors, I did not trust it blindly n actually counted the rows in my final output table. Am glad I checked. My by district rainfall export file listed only 56 total rows, not the 64 rows representing 8 districts multiplied by 8 years tht it should hve produced. I found a bad naming mistake inside my code setup. My MARATHWADA_DISTRICTS list used the standard common English spelling "Beed", but the old FAO GAUL 2015 dataset stores this specific area under the name "Bid" inside its ADM2_NAME column field. This caused a massive silent leak.

The filter line did not help. The ee.Filter.inList() command never throws a crash error when u type a spelling tht matches absolutely nothing on the grid. It jst drops the data quietly. My code silently dropped tht single district from my merged region boundary layer and deleted it from every single district level export across all

8 tracking years. It messed up my whole boundary shapefile logic. I chose to double check the real txt. Next I verified the proper spelling string against my raw district GeoJSON layer sitting inside my data/raw/ folder and fixed my python list text right away. This solved the problem. So I re ran my entire extraction script chain a 2nd time on my machine. Then I opened up my final output CSV file nd manually counted every line to make sure: 8 districts multiplied by 8 years gave me exactly 64 rows with all 8 names present on the sheet. Every downstream analysis code now uses this fully corrected table file.

So I ran my rainfall code next. I triggered rainfall_analysis.py against my corrected 8-year regional rainfall totals, n it gave me the most interesting result of this whole expansion pass tht I absolutely had not expected going in. This output really surprised me. I used the same drought threshold implied by my original study, which means roughly half a climatological standard deviation below the 2001–2020 mean value. My data showed that 2015 at −21.5% with z = −1.12 and 2018 at −18.3% with z = −0.95 remain the 2 absolute driest years, exactly like my original 3-year check stated. The other years looked very safe. None of my 5 newly added years even come close to tht dry line. My calculations showed 2016 hit +9.5%, 2017 got −1.1%, 2019 reached +13.3%, 2022 jumped to +36.8%, n 2023 sat at −3.7%, keeping them all within roughly 1 standard deviation of a normal season. This is a massive shift. So across my full 8-year record, only 2 out of 8 years actually qualify as real drought years by my own project definition. My original 3-year sample carried a 67% drought rate by pure chance of which years I picked 1st, which was way more drought heavy than Marathwada's real climate history. Will write this down plainly in my limitations section. It does not break my drought year findings at all, but it proves the original balance was unrepresentative of reality. My 8-year timeline gives a much clearer picture of wht's actually normal here.

I changed my hardcoded variables today. I switched every lazy DROUGHT_YEARS = {2015, 2018} literal string across my lag, cross correlation, nd bootstrap scripts to dynamically derive tht set straight from the actual rainfall anomaly file output instead of typing it by hand. It still resolves to the identical 2 years right now. But it means my classification is computed dynamically from the data going forward rather than being typed out manually like a novice script, which matters way more with 8 full years in play than it did with jst 3. This builds a much better pipeline structure.

Still pending as of this entry:

Still hve some massive tasks left on my queue. My clip_gosif.py and aggregate_sif.py scripts—the 2 pieces tht turn the raw GOSIF rasters into the actual working SIF time series data frames—need to run fully against all 216 raw files representing 27 composites multiplied by 8 years. This processing task has to happen locally on my own machine. Refuse to move them remotely purely because transferring 11+ gigabytes of raw heavy rasters over the network isn't worth moving anywhere. The clipped output size per file is actually under 40 kilobytes, so only tht tiny cropped text output needs to travel anywhere else for my dashboard or plots. Once tht long calculation pass is finished, all my SIF side findings will get reprocessed. I need to re run my lag analysis, cross correlation loops, bootstrap confidence intervals, Moran's I metrics, nd district level correlations against the full 8 years. My folder is stuck with the original 3-year numbers as of this log entry..... I hve a long night of data processing ahead.

Entry 16

I ran my main SIF processing tools. Executed clip_gosif.py nd zonal_stats_sif.py directly against all 216 raw GOSIF image files on my own machine, matching how I handled my download scripts back in Entry 15. Both files are fully generic. I used standard glob based file discovery strings with zero hardcoded year lists typed anywhere inside either script, so neither file required a single code edit to detect the 5 new data years. This

lazy design choice saved me. I deliberately wrote them this flexible way back when I was only tracking 3 years, nd it paid off exactly as intended here during this heavy expansion phase. It felt great to see it run smoothly.

My output tables came back looking fully populated. The marathwada_sif_timeseries.csv file populated with exactly 216 data rows representing 27 composites multiplied by 8 full years, and sif_by_district.csv generated 64 rows representing 8 districts multiplied by 8 years. These grid sizes matched my region level rainfall rows from Entry 15 down to the line. This served as an excellent manual sanity check tht my loop hadn't quietly dropped a year or skipped a district anywhere on the SIF processing side. Dataset's complete now.

I got the real 8-year SIF data in hand now. Reprocessed every single downstream analysis script against this big dataset, running compare_sif_ndvi.py, lag_analysis.py, cross_correlation_lag.py, bootstrap_lag_uncertainty.py, sif_rainfall_correlation.py, spatial_autocorrelation.py, n rq3_official_declaration_comparison.py on my terminal machine. It took a lot of processing hours... Also I changed my code setup to be more smart. I switched every remaining hardcoded DROUGHT_YEARS = {2015, 2018} txt string straight to the dynamic, rainfall anomaly derived classification version tht I already started coding back in Entry 15. Now the math is calculated uniformly everywhere. So I fixed 2 tiny deprecation warnings that popped up on my screen while working inside those script files, replacing the old matplotlib.cm.get_cmap line with matplotlib.colormaps[...].resampled(...) n swapping out geopandas's old .unary_union function with the modern .union_all() call. This cleanup pass made my entire project backend stable.

Found a huge 2018 exception. This is by far the single most interesting thing tht came out of my full data reprocessing pass. My scripts are telling a very weird story here. Every single lag estimation method—meaning threshold crossing calculations, cross correlation loops, nd the extra bootstrap check on top—all agree tht 2018 is acting pretty crazy compared to other years. I calculated its mean threshold crossing lag n it came out marginally negative at −1.1 days, which looks wild against the positive 4.2 to 23.8 days range for every other year in my sheet. It broke my pattern. Its cross correlation lag value is exactly zero. Also, only a tiny 8.1% of its bootstrap replicates show a positive lag score versus the high 42 to 97 percent seen for every other year in my study. So I checked it a 4th way using my RQ3 comparison against the official 2018 drought declaration paperwork. My data shows tht NDVI actually crosses its 90%-decline threshold *before* SIF does during tht specific year, which flips wht my original 3 year study reported early on. Am stuck — I simply do not have any specific crop type records, local sowing date files, or any other covariate data rows tht would explain why 2018 specifically behaves this way. So it just goes down as a plain, unexplained finding. I refuse to make up a fake guess or quietly average it into my drought year group jst to keep my supervisor happy. That's this entry wrapped up.

I found my other headline number today. Only 2 out of my full 8 timeline years are actually real, documented drought years on my spreadsheet. Already flagged this issue back in Entry 15 from the pure rainfall deficit side alone, nd now it holds up identically once I bring my SIF dataset layers into the picture too. My original selection was highly skewed. My old 3 year sample being 2/3 drought years was, in hindsight, a considerably more drought heavy selection than Marathwada's actual 8 year historical climate record supports. Had a wrong picture of my home region early on.

Weaker, but still real, at the larger sample.

My district level SIF rainfall correlation dropped significantly. I calculated that the r value fell from my old r = 0.837 where n = 24 all the way down to r = 0.567 where n = 64 across the full sheets. My math is still highly significant. This is a solid, clear weakening of the signal strength rather than some tiny rounding difference, n I am choosing to report it as exactly tht instead of loudly emphasizing the good p value and burying the drop.

My Moran's I spatial calculations tell a similarly 2 sided story. I found tht rainfall's spatial clustering stayed significant across all 8 working years, matching the 3-for-3 pattern I originally calculated. But my SIF results acting weird now — its spatial clustering is now significant in only 4 out of my 8 years on the map grid. It is real in some years but undetectable in others. This gives me a much more qualified, messy picture than my original all 3 years significant result implied. Logs are caught up now.

Had to fully rewrite my district level ranking section. My new 8 year calculations proved tht Nanded, which used to be the highest SIF district in all 3 of my original baseline years, only leads in 2 out of 8 years now. My older layout logic crashed. Found that Aurangabad actually leads most of the time sitting at 4 out of 8 years, while 2 other random districts steal the top lead spot exactly once each. This is a different story. The consistently highest framing statement from my original 3 year study simply does not hold true at this much larger sample size. So, I chose to say so directly in my journal. I refused to jst lazily calculate which district happened to average out highest over the decade and call it consistent for my paper presentation. Wanted this to reflect wht's actually there, not a tidier story.

The bottom of my ranking chart looks way more stable. I found tht Osmanabad stays stuck at the absolute lowest SIF rank in 7 out of my 8 working years. It is a highly consistent hotspot. But Bid takes over tht lowest spot specifically in the year 2018. I checked why this happened. This is the same district tht recorded the single lowest SIF value of my entire study, nd it also matches the year's worst recorded rainfall deficit. My old timeline claims broke down too, since my original claim that every single year's lag is strictly positive is no longer true at 8 years because 4 years now tie at exactly zero days. This is a massive shift.

This log entry finally closes out my full timeline sample size expansion task. Every single number inside my project folder is now the real, updated 8 year result, not a messy partially updated mix of old nd new data lines. Spreadsheets finally match up across the board.

Entry 17

Found a real bug today, nd it's not a small one. Was doing another pass over cross_correlation_lag.py nd bootstrap_lag_uncertainty.py, jst reading the code slowly line by line and not trusting tht it already worked. My stomach dropped a bit when I saw it. Both scripts search for the lag tht best lines up SIF nd NDVI, but the search range was written as `np.arange(0, max_lag + 1)` in both files. The search only ever checked lags from zero upward, meaning it was mathematically impossible for either script to ever come back with NDVI leading SIF, no matter wht the actual satellite data showed. It's not tht the data said "never negative." It's tht I never let the code look there.

This changes wht "SIF's lag is never negative" actually means. Every earlier entry in this log, nd the paper itself up till now, reported the cross correlation lag as never negative n the bootstrap as showing 100% of replicates non negative in every year. Both of those statements were technically true, but only because I'd built a search space tht ruled out the other answer before it ever looked at a single number. Tht's not a finding, tht's a coin flip I rigged by accident. Felt pretty bad realizing this sat in the paper for this long.

Fixed it properly rather than jst patching numbers. In cross_correlation_lag.py, changed `lags = np.arange(0, max_lag + 1)` to `lags = np.arange(-max_lag, max_lag + 1)`, nd added a small `lagged_corr()` helper tht handles positive lag, negative lag, nd zero lag as 3 separate cases since numpy slicing with a negative shift needed to go the other direction. Did the same 2 sided fix inside bootstrap_lag_uncertainty.py's `lag_from_daily()` function, nd updated its boundary safety check (`abs(lag) >= len(sif_c) - 4`) so it protects against running out of overlapping points on the negative side too, not jst the positive side like before. Reran both scripts from scratch on the real merged SIF/NDVI data.

The corrected numbers tell a whole different story, not jst a wording fix. Cross correlation lag by year now reads: 2015 at 4 days, 2016 at 2 days, 2017 at 27 days, 2018 at −4 days, 2019 at 18 days, 2020 at 0 days, 2022 at −9 days, nd 2023 at −10 days. Under the old 1 sided search, 2018, 2020, 2022, nd 2023 all landed at exactly zero because zero was the best the search was allowed to find — now 3 of those 4 years, 2018, 2022, nd 2023, come out clearly negative, meaning NDVI actually leads SIF in those years by this method. Only 2020 stays a true tie at zero. So instead of "SIF leads in half the years, ties in the other half," it's now "SIF leads in 4 years, ties in 1, nd NDVI leads in 3."

The bootstrap confirms this isn't noise. 2022 nd 2023 now show 99.4% nd 99.1% of their replicates landing below zero — tht's a strong, well resolved signal in the NDVI leads direction, not a coin flip. 2018 leans the same way at 87.4% of replicates negative, though its confidence interval still straddles zero so it's less settled than 2022/2023. Redid the pairwise year comparison too. Of the 28 possible year pairs, 24 still overlap nd aren't statistically distinguishable, but 4 pairs now clearly don't overlap — 2015 vs 2022, 2015 vs 2023, 2017 vs 2022, nd 2017 vs 2023. So the 2 clearest SIF leads years are now statistically distinguishable from the 2 clearest NDVI leads years. Tht's a real result the old 1 sided search could never have produced, since it wasn't allowed to put any year on the NDVI side to begin with.

Went through the whole project after this nd fixed every place tht repeated the old "never negative" framing. Updated GA_Research_Paper.md's methodology, results, abstract, discussion, limitations, nd conclusion sections, regenerated both figures — cross_correlation_lag.png nd cross_correlation_lag_bootstrap_ci.png — updated the 2 dashboard pages tht show these numbers, GA_Executive_Summary.md, nd README.md. Disclosed the bug directly everywhere and not quietly swapping in new numbers, same as I did for the Beed/Bid boundary bug nd the Earth Engine Cloud project bug earlier in this log. This one actually changes a real scientific finding, not jst a data processing detail, so it gets fixed the same way, out in the open.

While I was in there I also found a small leftover from Entry 16 tht I said I'd already fixed but hadn't — clip_gosif.py still had the old `boundary.geometry.unary_union` call instead of `union_all()`. I thought I swapped this everywhere back in Entry 16 but missed this one file. Fixed it now too, small thing, but it's going in the log rather than pretending Entry 16 was complete when it wasn't.

This was not a fun bug to find in my own project this late, but I'm glad I caught it before EarthArXiv and not after. My "no compromises" rule from the very first line of this log means this too. A wrong search range that quietly guarantees its own answer is exactly the kind of mistake tht rule exists to catch. Reported it in full rather than burying it in a changelog nobody reads.